Deep Learning - lecture 3

Non-linear models

Prof. Stefano Carrazza

University of Milan and INFN Milan

Artificial neural networks

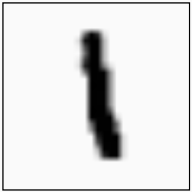
Limitations of linear models

Why not linear models everywhere?

Limitations of linear models

Why not linear models everywhere?

Example: consider 1 image from the MNIST database:



$$\mathbb{R}^{\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}}$$

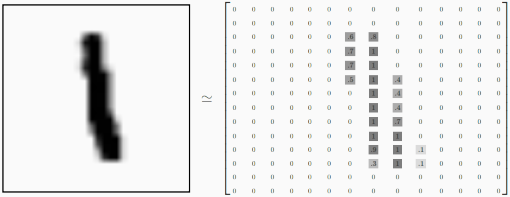
Each image has 28x28 pixels $=$ 785 features (x3 if including RGB colors).

If consider quadratic function $\mathcal{O}(n^2)$ so linear models are impractical.

Limitations of linear models

Why not linear models everywhere?

Example: consider 1 image from the MNIST database:

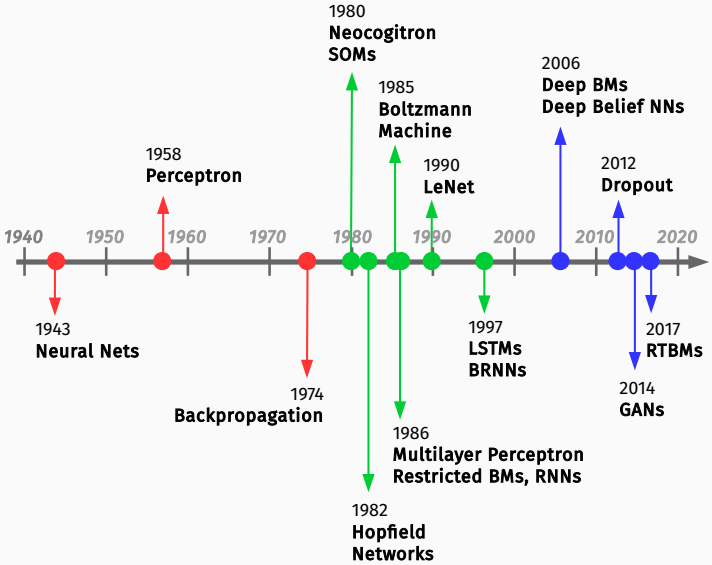


Each image has 28x28 pixels = 785 features (x3 if including RGB colors).

If consider quadratic function $\mathcal{O}(n^2)$ so linear models are impractical.

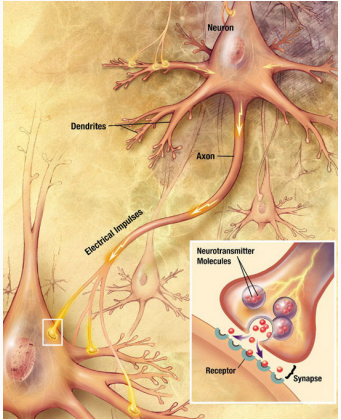
Solution: use non-linear models.

Non-linear models timeline



Neural networks

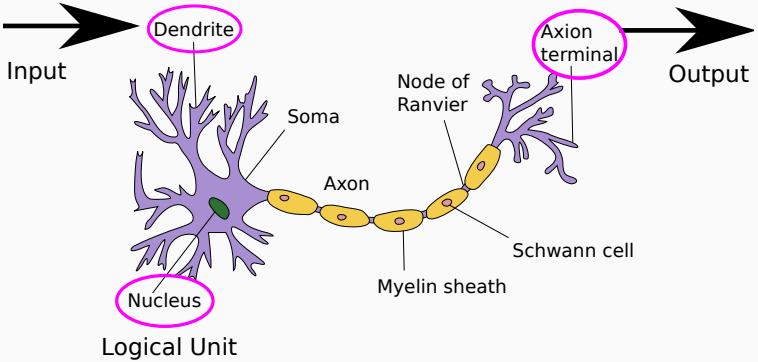
Artificial neural networks are computer systems inspired by the biological neural networks in the brain.



Currently the state-of-the-art technique for several ML applications.

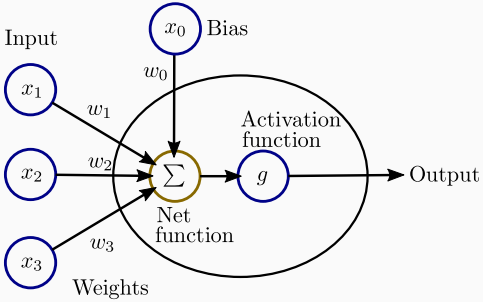
Neuron model

We can imagine the following data communication pattern:



Neuron model

Schematically:



where

- each **node** has an associate weights and bias w and inputs x,
- the output is modulated by an **activation function**, g.

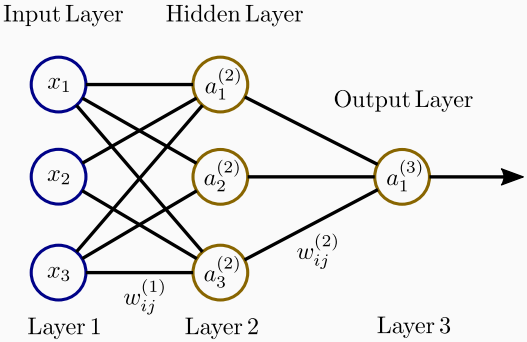
Some examples of activation functions: sigmoid, tanh, linear, ...

$$g_w(x) = \frac{1}{1 + e^{-w^T x}}, \quad \tanh(w^T x), \quad x.$$

Neural networks

In practice, we simplify the bias term with $x_0 = 1$.

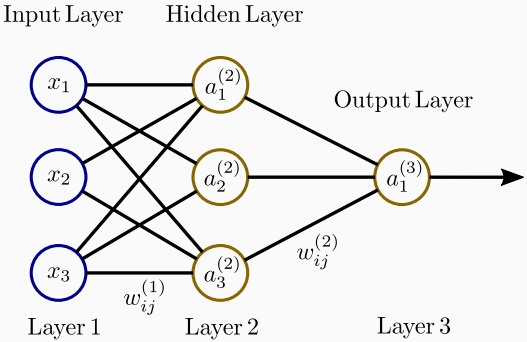
Neural network $\rightarrow$ connecting multiple units together.



where

- $a_i^{(l)}$ is the activation of unit i in layer l,
- $w_{ij}^{(l)}$ is the weight between nodes i, j from layers $l, l+1$ respectively.

Neural networks

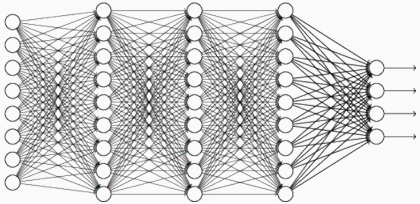


- $a_1^{(2)} = g(w_{10}^{(1)} + w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + w_{13}^{(1)} x_3)$
- $a_2^{(2)} = g(w_{20}^{(1)} + w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + w_{23}^{(1)} x_3)$
- $a_3^{(2)} = g(w_{30}^{(1)} + w_{31}^{(1)} x_1 + w_{32}^{(1)} x_2 + w_{33}^{(1)} x_3)$
- **Output** $\rightarrow a_1^{(3)} = g(w_{10}^{(2)} + w_{11}^{(2)} a_1^{(2)} + w_{12}^{(2)} a_2^{(2)} + w_{13}^{(2)} a_3^{(2)})$

Neural networks

Some useful names:

- Feedforward neural network: no cyclic connections between nodes from the same layer (previous example).
- Multilayer perceptron (MLP): is a feedforward neural network with at least 3 layers.
- Deep neural networks: term referring to neural networks with more than one hidden layer.



Training neural networks

The training NNs is usually performed with gradient descent methods.

Following the previous section, we have to compute the cost function gradient with respect to parameters $w_{ij}^{(l)}$:

$$w_{ij}^{(l)} := w_{ij}^{(l)} - \eta \nabla_{ij}^{(l)} J \quad \rightarrow \quad \nabla_{ij}^{(l)} J = \frac{\partial}{\partial w_{ij}^{(l)}} J(\boldsymbol{w})$$

Training neural networks

The training NNs is usually performed with gradient descent methods.

Following the previous section, we have to compute the cost function gradient with respect to parameters $w_{ij}^{(l)}$:

$$w_{ij}^{(l)} := w_{ij}^{(l)} - \eta \nabla_{ij}^{(l)} J \quad \rightarrow \quad \nabla_{ij}^{(l)} J = \frac{\partial}{\partial w_{ij}^{(l)}} J(\boldsymbol{w})$$

Use the backpropagation algorithm to compute the gradient of a NN.

- can be used with any gradient-based optimizer, including quasi-Newton methods.
- reduces the large amount of computations thanks to chain rule
- requires the derivative of the cost function with respect to the output layer $w_{ij}^{(l)}$ with $l =$ output.

Backpropagation algorithm

The backpropagation steps:

1: perform a forward propagation (calculate $a_i^{(l)}$)

Backpropagation algorithm

The backpropagation steps:

1: perform a forward propagation (calculate $a_i^{(l)}$)

2: perform a backward propagation: evaluate for each node a "prediction error":

$$\delta_j^{(l)} = \text{“error” of node } j \text{ in layer } l.$$

Backpropagation algorithm

The backpropagation steps:

1: perform a forward propagation (calculate $a_i^{(l)}$)

2: perform a backward propagation: evaluate for each node a "prediction error":

$$\delta_j^{(l)} = \text{“error” of node } j \text{ in layer } l.$$

3: calculate $\nabla_{ij}^{(l)} J$ using erros $\delta_i^{(l)}$ and $a_i^{(l)}$.

Backpropagation algorithm

The backpropagation steps:

1: perform a forward propagation (calculate $a_i^{(l)}$)

2: perform a backward propagation: evaluate for each node a "prediction error":

$$\delta_j^{(l)} = \text{“error” of node } j \text{ in layer } l.$$

3: calculate $\nabla_{ij}^{(l)} J$ using erros $\delta_i^{(l)}$ and $a_i^{(l)}$.

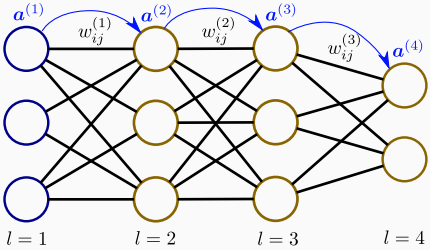
4: perform weight updates, $\Delta w_{ij}^{(l)}$, via gradient descent using $\nabla_{ij}^{(l)} J$.

Backpropagation algorithm

Suppose we have a MLP and one training example $(\boldsymbol{x}, \boldsymbol{y})$.

Step 1: We first perform a forward propagation pass:

- $\boldsymbol{a}^{(1)} = \boldsymbol{x}$

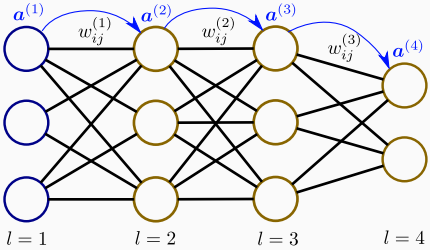


Backpropagation algorithm

Suppose we have a MLP and one training example $(\boldsymbol{x}, \boldsymbol{y})$.

Step 1: We first perform a forward propagation pass:

- $\boldsymbol{a}^{(1)} = \boldsymbol{x}$
- $\boldsymbol{z}^{(2)} = \boldsymbol{w}^{(1)}\boldsymbol{a}^{(1)}$

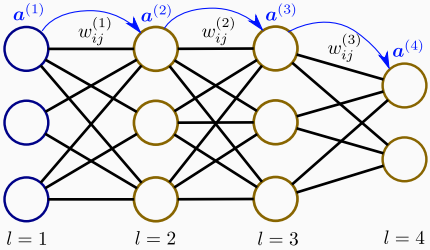


Backpropagation algorithm

Suppose we have a MLP and one training example $(\boldsymbol{x}, \boldsymbol{y})$.

Step 1: We first perform a forward propagation pass:

- $\boldsymbol{a}^{(1)} = \boldsymbol{x}$
- $\boldsymbol{z}^{(2)} = \boldsymbol{w}^{(1)}\boldsymbol{a}^{(1)}$
- $\boldsymbol{a}^{(2)} = g(\boldsymbol{z}^{(2)})$

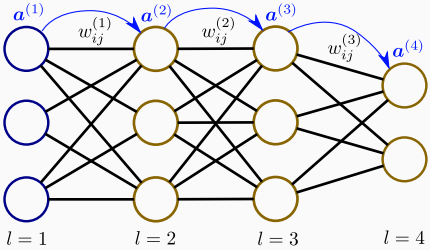


Backpropagation algorithm

Suppose we have a MLP and one training example $(\boldsymbol{x}, \boldsymbol{y})$.

Step 1: We first perform a forward propagation pass:

- $\boldsymbol{a}^{(1)} = \boldsymbol{x}$
- $\boldsymbol{z}^{(2)} = \boldsymbol{w}^{(1)}\boldsymbol{a}^{(1)}$
- $\boldsymbol{a}^{(2)} = g(\boldsymbol{z}^{(2)})$
- $\boldsymbol{z}^{(3)} = \boldsymbol{w}^{(2)}\boldsymbol{a}^{(2)}$

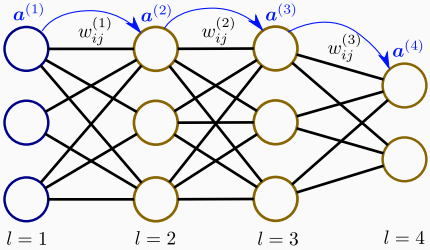


Backpropagation algorithm

Suppose we have a MLP and one training example $(\boldsymbol{x}, \boldsymbol{y})$.

Step 1: We first perform a forward propagation pass:

- $\boldsymbol{a}^{(1)} = \boldsymbol{x}$
- $\boldsymbol{z}^{(2)} = \boldsymbol{w}^{(1)}\boldsymbol{a}^{(1)}$
- $\boldsymbol{a}^{(2)} = g(\boldsymbol{z}^{(2)})$
- $\boldsymbol{z}^{(3)} = \boldsymbol{w}^{(2)}\boldsymbol{a}^{(2)}$
- $\boldsymbol{a}^{(3)} = g(\boldsymbol{z}^{(3)})$

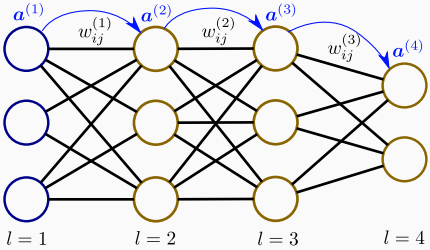


Backpropagation algorithm

Suppose we have a MLP and one training example $(\boldsymbol{x}, \boldsymbol{y})$.

Step 1: We first perform a forward propagation pass:

- $\boldsymbol{a}^{(1)} = \boldsymbol{x}$
- $\boldsymbol{z}^{(2)} = \boldsymbol{w}^{(1)}\boldsymbol{a}^{(1)}$
- $\boldsymbol{a}^{(2)} = g(\boldsymbol{z}^{(2)})$
- $\boldsymbol{z}^{(3)} = \boldsymbol{w}^{(2)}\boldsymbol{a}^{(2)}$
- $\boldsymbol{a}^{(3)} = g(\boldsymbol{z}^{(3)})$
- $\boldsymbol{z}^{(4)} = \boldsymbol{w}^{(3)}\boldsymbol{a}^{(3)}$

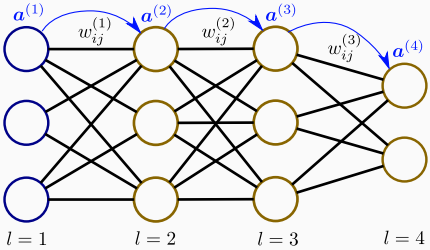


Backpropagation algorithm

Suppose we have a MLP and one training example $(\boldsymbol{x}, \boldsymbol{y})$.

Step 1: We first perform a forward propagation pass:

- $\boldsymbol{a}^{(1)} = \boldsymbol{x}$
- $\boldsymbol{z}^{(2)} = \boldsymbol{w}^{(1)}\boldsymbol{a}^{(1)}$
- $\boldsymbol{a}^{(2)} = g(\boldsymbol{z}^{(2)})$
- $\boldsymbol{z}^{(3)} = \boldsymbol{w}^{(2)}\boldsymbol{a}^{(2)}$
- $\boldsymbol{a}^{(3)} = g(\boldsymbol{z}^{(3)})$
- $\boldsymbol{z}^{(4)} = \boldsymbol{w}^{(3)}\boldsymbol{a}^{(3)}$
- Output $\boldsymbol{a}^{(4)} = g(\boldsymbol{z}^{(4)})$



At this step we know the output of the current MLP setup.

Backpropagation algorithm

2. evaluate for each node the error $\delta_j^{(k)}$ for $k = 2, 3, \ldots, L$.

Some remarks:

It is possible to proof using derivative chain rules that:

$$\nabla_{ij}^{(l)} J = \frac{\partial J}{\partial z_i^{(l+1)}} a_j^{(l)} \equiv \delta_i^{(l+1)} a_j^{(l)},$$

for $l = 1, \ldots, L-1$.

Backpropagation algorithm

2. evaluate for each node the error $\delta_j^{(k)}$ for $k = 2, 3, \ldots, L$.

Some remarks:

It is possible to proof using derivative chain rules that:

$$\nabla_{ij}^{(l)} J = \frac{\partial J}{\partial z_i^{(l+1)}} a_j^{(l)} \equiv \delta_i^{(l+1)} a_j^{(l)},$$

for $l = 1, \ldots, L-1$.

The recursive relation for the error is:

$$\delta_i^{(l)} = \sum_k w_{ki}^{(l)} \delta_k^{(l+1)} \cdot g'(z_i^{(l)})$$

and at $l = L$, *i.e.* the highest l index:

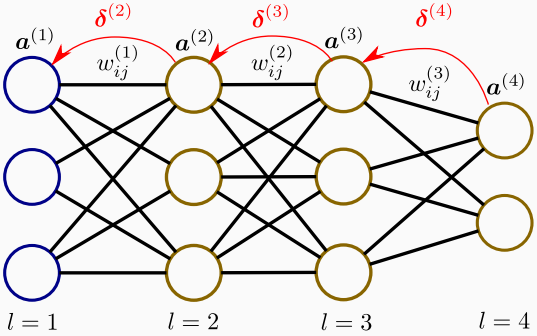
$$\delta_i^{(L)} = \frac{\partial J}{\partial a_i^{(L)}} \cdot g'(z_i^{(L)})$$

where $g'(z_i^{(l)}) = a_i^{(l)}(1 - a_i^{(l)})$ if g is the sigmoid function.

Backpropagation algorithm

Example: evaluating error $\delta_j^{(l)}$ for a MLP with sigmoids in the hidden layers and linear activation function in the output layer:

- $\boldsymbol{\delta}^{(4)} = \boldsymbol{a}^{(4)} - \boldsymbol{y}$
- $\boldsymbol{\delta}^{(3)} = (\boldsymbol{w}^{(3)})^T \boldsymbol{\delta}^{(4)} \cdot (\boldsymbol{a}^{(3)}(1 - \boldsymbol{a}^{(3)}))$
- $\boldsymbol{\delta}^{(2)} = (\boldsymbol{w}^{(2)})^T \boldsymbol{\delta}^{(3)} \cdot (\boldsymbol{a}^{(2)}(1 - \boldsymbol{a}^{(2)}))$



Backpropagation algorithm summary

Data: training set $(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)})$ with $i = 1, \ldots, m$ examples.

Result: the trained neural network

Initialize network weights;

while *stopping criterion is not satisfied* **do**

  Set all $\Delta w_{ij}^{(l)} = 0$.

  for $k = 1$ **to** m **do**

    Perform forward pass and compute $\boldsymbol{a}^{(l)}$ for $l = 1, 2, 3, \ldots, L$;

    Perform backward pass and compute $\boldsymbol{\delta}^{(l)}$ for $l = 2, \ldots, L$;

    $\Delta w_{ij}^{(l)} := \Delta w_{ij}^{(l)} + a_j^l \delta_i^{(l+1)}$

  end

  Update network weights using gradient descent;

end

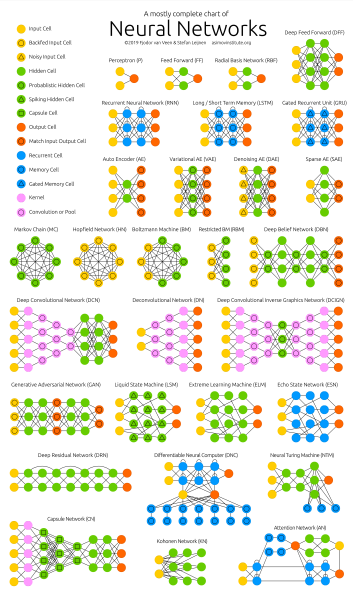
Training neural networks

Some remarks and example of neural network initialization:

- zero: all weights are set to zero so all neurons perform the same calculation. The complexity of the neural network is equivalent to a single neuron.
- random: breaks parameter symmetry.
- glorot/xavier: initialize each weight with a small Gaussian value with mean zero and variance based on the in/out size of the weight.
- he: avoid activation function saturation. Weights are random initialized considering the size of the previous layer.

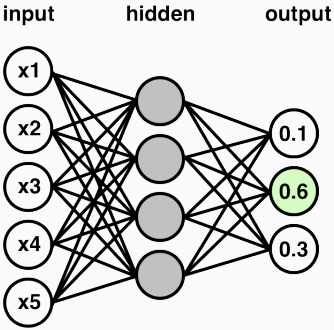
Neural networks zoo

Artificial neural networks architectures



MLP

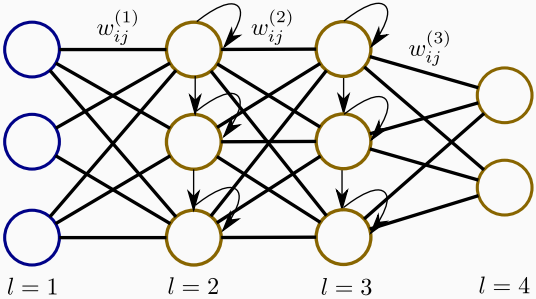
- Sequential model (MLP): regression and classification



Artificial neural networks architectures

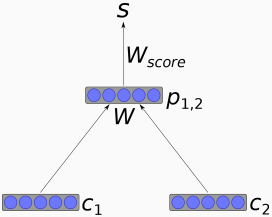
Some examples of neural network popular architectures:

- Recurrent neural networks: neural networks where connections between nodes form a directed cycle.
  - built-in internal state memory
  - built-in notion of time ordering for a time sequence



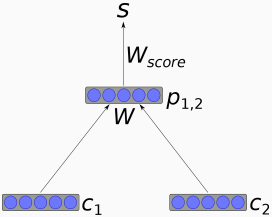
Artificial neural networks architectures

- Recursive neural networks: a variation of recurrent neural network where pairs of layers or nodes are merged recursively.
  - successful applications on natural language processing.
  - some recent applications for model inference.



Artificial neural networks architectures

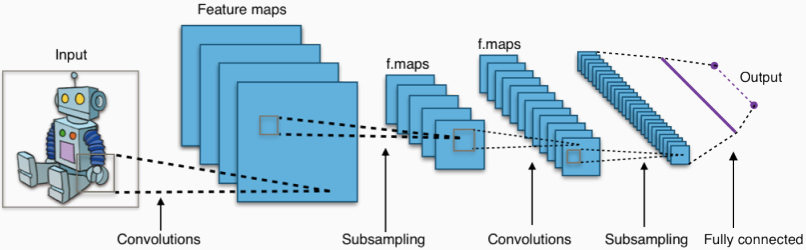
- Recursive neural networks: a variation of recurrent neural network where pairs of layers or nodes are merged recursively.
  - successful applications on natural language processing.
  - some recent applications for model inference.



- Long short-term memory: another variation of recurrent neural networks composed by custom units cells:
  - LSTM cells have an input gate, an output gate and a forget gate.
  - powerful when making predictions based on time series data.

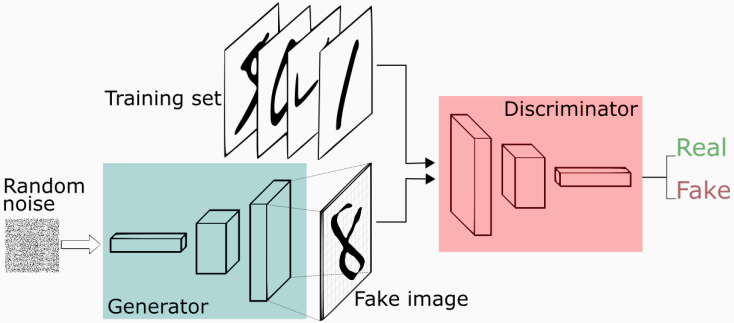
Artificial neural networks architectures

- Convolutional neural networks: multilayer perceptron designed to require minimal preprocessing, *i.e.* space invariant architecture.
  - the hidden layers consist of convolutional layers, pooling layer, fully connected layers and normalization layers
  - great successful applications in image and video recognition.



Artificial neural networks architectures

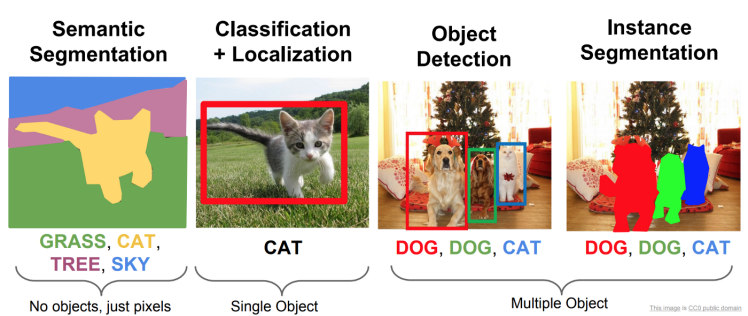
- Generative adversarial network: unsupervised machine learning system of two neural networks contesting with each other.
  - one network generate candidates while the other discriminates.



Digital images treatment

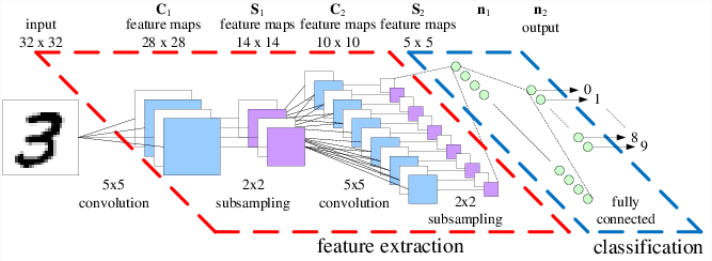
Pattern recognition algorithms:

- **semantic segmentation:** predict labels for pixels
- **classification + localization:** predict object and its location
- **object detection:** identify multiple objects and locations
- **instance segmentation:** identify multiple objects and masks



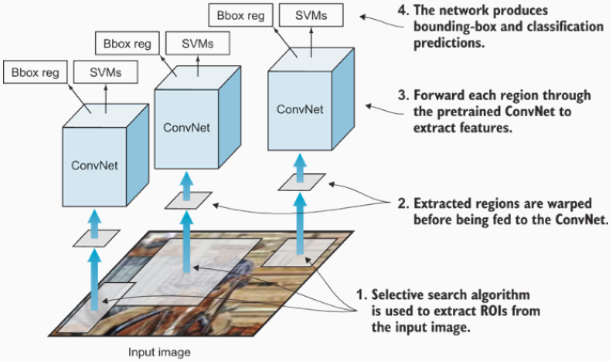
Models for digital image treatment

- CNN: image classification



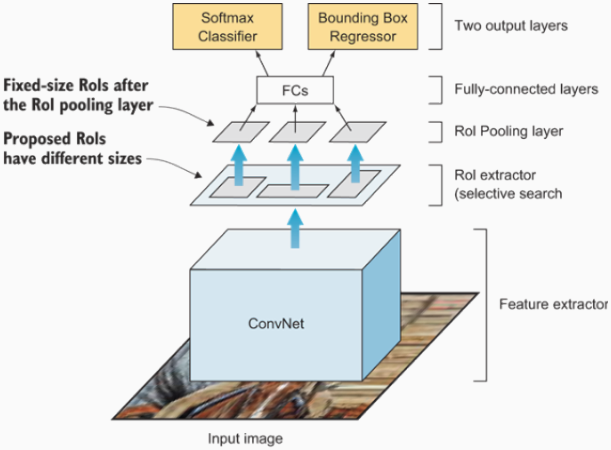
Models for digital image treatment

- R-CNN: object detection



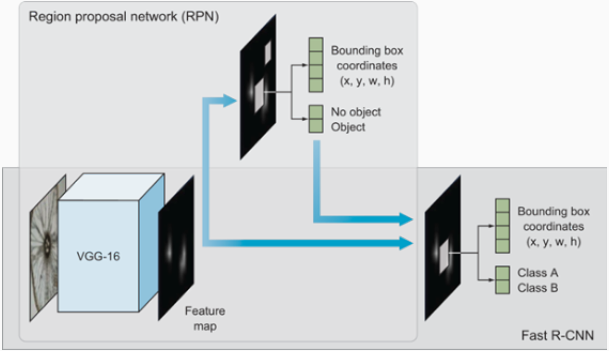
Models for digital image treatment

- Fast R-CNN: object detection



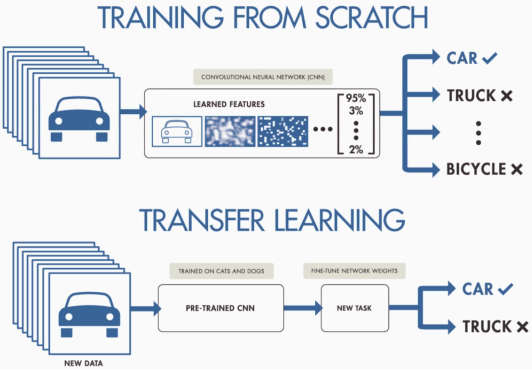
Models for digital image treatment

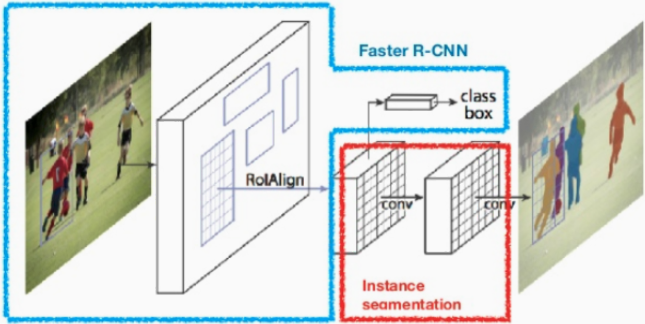
- Faster R-CNN: object detection



Models for digital image treatment

Transfer learning: a research problem in machine learning (ML) that focuses on storing knowledge gained while solving one problem and applying it to a different but related problem.



Models for digital image treatment

- Mask-RCNN or UNet: instance segmentation